# REFINE

# Users Guide

## Version 6.23

**written by**

**Reinhard Neder**
Email: reinhard.neder@fau.de

**http://tproffen.github.io/DiffuseCode**

*Document created: September 6, 2026*

# Preface

## Disclaimer

The REFINE software described in this guide is provided without warranty of any kind. No liability is taken for any loss or damages, direct or indirect, that may result through the use of *REFINE*. No warranty is made with respect to this manual, or the program and functions therein. There are no warranties that the programs are free of error, or that they are consistent with any standard, or that they will meet the requirement for a particular application. The programs and the manual have been thoroughly checked. Nevertheless, it can not be guaranteed that the manual is correct and up to date in every detail. This manual and the REFINE program may be changed without notice.
REFINE is intended as a public domain program. It may be used free of charge. Any commercial use is, however, not allowed without permission of the authors.

## Using REFINE

## More information

This users guide can only provide program specific details. A broader discussion of simulation techniques and some REFINE examples and macro files can be found in our book

> NEDER, R.B. & PROFFEN, TH. "Diffuse Scattering and Defect Structure Simulations - A cook book using the programs DISCUS", *IUCr Texts on Crystallography*, Oxford University Press, 2007.

# Contents

**1 Introduction** **4**
1.1 What is REFINE ? . . . 4
1.2 Getting started . . . 4
1.3 Command language . . . 6

**2 Least Squares Refinement** **7**
2.1 Refinement via least squares refinement . . . 7
2.2 Algorithm in the REFINE section . . . 8

**3 Example refinements** **11**
3.1 Overview . . . 11
3.2 Simple noisy data function . . . 11
3.3 Nanoparticle PDF . . . 15
3.4 Single crystal refinement . . . 20

**4 Creating a DISCUS SUITE refinement Macro Set** **25**
4.1 Setting up a single crystal Bragg refinement . . . 27
4.2 Setting up a single crystal refinement . . . 27
4.3 Setting up a powder diffraction refinement . . . 28
4.4 Setting up a PDF refinement . . . 29
4.5 Adding disorder to a refinement . . . 29

**5 Mathematical Background** **30**
5.1 Least squares algorithm . . . 30
5.1.1 Linear least squares algorithm . . . 31
5.1.2 Non-linear least squares algorithm . . . 33

**A REFINE commands** **36**
A.1 Summary . . . 36
A.2 News : Information on program updates . . . 36
A.3 Example . . . 36
A.4 manual . . . 37
A.5 data . . . 37
A.6 finished . . . 37
A.7 fix . . . 38

A.8 newparam . . . . . 38
A.9 reset . . . . . 39
A.10 run . . . . . 39
A.11 show . . . . . 39
A.12 set . . . . . 39
A.13 sigma . . . . . 41

**Bibliography** **42**

# Chapter 1

# Introduction

## 1.1 What is REFINE ?

REFINE is the direct Least Squares Refinement section of the DISCUS SUITE program. Its internal engine is a Levenberg-Marquardt type least squares fit that can be applied to 1D or 2D data.
See the DIFFEV section on a more general generic evolutionary refinement program that implements the differential evolutionary algorithm Price et al. (2005). Evolutionary or genetic refinement algorithms allow the refinement of models, functions, or more generally speaking the parameters of a cost function to obtain a good solution.
A least squares based refinement of a function $y = F(p_0, p_1, ..., p_n)$ requires the calculation of all partial derivatives $\partial y/\partial p_i$, either from an analytical or a numeric solution. The refinement that is described in this section is intended to work with disordered crystal structures as build by DISCUS . To achieve this REFINE uses a macro that is provided by the user to build a crystal and to calculate a diffraction pattern or a PDF. As the details of a calculation are hidden within the source code of the DISCUS and KUPLOT section, REFINE generally relies on a numerical calculation of the derivative.
Since this macro could calculate any cost function, REFINE is not limited to the refinement of a particular physical problem.

## 1.2 Getting started

After the program *DISCUS_SUITE* is installed properly and the environment variables are set, the program can be started by typing 'discus_suite' at the operating systems prompt.
The section uses the identical command language to interact with the user as is used throughout the DISCUS SUITE. The command `exit` terminates the section and returns control to the top level of the DISCUS SUITE. All commands of REFINE consist of a command verb, optionally followed by one or more parameters. All parameters must be separated from one another by a comma ",". There is no predefined need for any specific sequence of commands. REFINE is case sensitive, all commands and alphabetic parameters MUST be typed in lower case letters. If REFINE has been compiled using the `-DREADLINE` option (see installation files) basic line editing and recall of commands is possible. For more information refer to the reference man-

| Symbol | Description |
|---|---|
| "text" | Text given in double quotes is to be understood as typed. |
| <text> | Text given in angled brackets is to be replaced by an appropriate value, if the corresponding line is used in DIFFEV. It could, for example be the actual name of a file, or a numerical value. |
| `text` | Text in single quotes exclusively refers to REFINE commands. |
| [text] | Text in square brackets describes an optional parameter or command. If omitted, a default value is used, else the complete text given in the square brackets is to be typed. |
| {text \| text} | Text given in curly brackets is a list of alternative parameters. A vertical line separates two alternative, mutually exclusive parameters. |

**Table 1.1:** Used symbols

| Variable | Description |
|---|---|
| F_DATA | Kuplot data set that holds the experimental data. |
| F_SIGMA | Kuplot data set that holds the experimental uncertainties. |
| F_XMIN | Minimum 'x'-value of the experimental data set. |
| F_XMAX | Maximum 'x'-value of the experimental data set. |
| F_XSTP | Interval size along the 'x-axis of the experimental data set. |
| F_YMIN | Minimum 'y'-value of the experimental data set. xyz data only |
| F_YMAX | Maximum 'y'-value of the experimental data set. xyz data only |
| F_YSTP | Interval size along the 'y-axis of the experimental data set. xyz data only |

**Table 1.2:** REFINE variables. Variables marked with * are read-only and cannot be altered.

ual or check the online help using (`help command input`). Names of input or output files are to be typed as they will be expected by the shell. If necessary include a path to the file. All commands may be abbreviated to the shortest unique possibility. At least a single space is needed between the command verb and the first parameter. No comma is to precede the first parameter. A line can be marked as comment by inserting a "#" as first character in the line.
The symbols used throughout this manual to describe commands, command parameters, or explicit text used by the program REFINE are listed in Table 1.1. There are several sources of information, first REFINE has a build in online help, which can be accessed by entering the command `help` or if help for a particular command <cmd> is wanted by `help <cmd>`. This manual describes background and principle functions of REFINE and should give some insight in the ways to use this program.
REFINE is distributed as part of the diffuse scattering simulation software DISCUS. However, REFINE can be used as general refinement program separate from the main purpose of the DISCUS program package.

## 1.3 Command language

The program includes a FORTRAN style interpreter that allows the user to program complex modifications. A detailed discussion about the command language, which is common to all DISCUS package programs can be found in the separate DISCUS package reference guide which is included with the package.
Table 1.2 shows a summary of REFINE specific variables. All variables are read/write.

# Chapter 2

# Least Squares Refinement

## 2.1 Refinement via least squares refinement

Every time we measure some physical effect and wish to understand how this effect works, we want to determine the parameters of a model function that will replicate the observations. The term refinement refers to the process by which the parameters of the function are tuned such as to give the best agreement between the observed and calculated values. The term *best agreement* merits careful definition, for right now it is sufficient to say that the sum over all squared differences between the observations and the calculations shall be minimized. Thus, refinement is but a special case of general optimization. A very different example for an optimization could be the task to place as many integrated circuits into a chip and simultaneously achieve the fastest computations. Quite well known is the traveling salesman problem. Here the optimization task requires to find the shortest route that visits a number of spots distributed in space.
By far the fastest refinement technique is a least squares algorithm. Such an algorithm can always be applied if we can describe the physical effect as a function of parameters:

$$y \; = \; F(p_0, p_1, ..., p_n), \tag{2.1}$$

and all the partial derivatives $\partial y / \partial p_i$ can be calculated, either analytically or numerically. For each observed value $y_{obs}$, we calculate a value $y_{calc}$ and minimize the value of a weighted residual wR:

$$wR = \sqrt{\frac{\sum_i w_i (y_{obs}(i) - y_{calc}(i))^2}{\sum_i w_i y_{obs}(i)^2}} \tag{2.2}$$

Here each difference is multiplied by a weight $w_i$ that reflects the uncertainties of the experimental values. In case of crystal structure analysis, the observed values would be the observed intensities in a diffraction pattern and the calculated values those intensities that were calculated based on a structural model. Model parameters will be the lattice parameters, the positions of the atoms in the unit cell, atomic displacement parameters etc. as well as experimental parameters, such as the background. Under the assumption that we have a periodic crystal, the partial derivatives of the intensity with respect to lattice parameters, atom positions etc., can all be derived analytically. This is the concept you will find within any single crystal structure refinement program or a Rietveld program.

For disordered structures, the situation becomes more complicated. Except for a few special cases like stacking faults or short-range order problems, no general analytical function straightforwardly links the disorder parameter to the intensity. The intensity can still be calculated from structural models. The simulation, however, usually involves the application of random choices to generate part or all of the atom positions, and the analytical derivative of the intensity with respect to the order parameter is no longer available. A numeric calculation of the derivatives involves the repeated simulation of a new model for each parameter and is fairly time consuming.

## 2.2 Algorithm in the REFINE section

The REFINE section uses a Least-Squares algorithm based on the software in the Numerical Recipes Press et al. (1989). This software is based on the Levenberg-Marquart algorithm. In any least-squares algorithm the derivatives are used to determine a new estimate for each parameter. While the refinement is far from the final solution, each parameter can be modified by a large step in order to quickly approach the global minimum. Close to the minimum the steps need to be smaller not to miss the minimum. Essentially the step size is roughly propertional to the derivative. The Levenberg-Marquardt algorithm, optimizes the steps to be taken.
Within the REFINE section of the DISCUS SUITE the derivatives are calculated numerically. The program runs the simulation several times for each parameter: at the current parameter value and at slightly larger and slightly smaller parameter values. The resulting R-values at each parameter value are analyzed to calculate the derivative for this parameter. The REFINE section calculates the R-value for a given parameter at:

| | |
|---|---|
| P | R-value(P) |
| P+h | R-value(P+h) |
| P-h | R-value(P-h) |
| P+2h | R-value(P+2h) |
| P-2h | R-value(P-2h) |

The REFINE section calculates a polynomial of order two through these three points and uses this polynomial to derive the value of the derivative a the current value of the Parameter `P`.
In any numerical determination of the derivative it is not straightforward to know what is the best value of the deviation `h` from the current parameter value `P`. If the function whose derivative we need to obtain is a smooth function, a very small value of `h` is best, as this is likely to yield a good approximation to the analytical derivative. The value `h` must, however, be large enough to produce a R-value that differs significantly from the original R-value and whose calculation is not affected by rounding errors that are unavoidable for very small numerical values. Within the REFINE section the value of the absolute shift `h` of the parameter `P` is calculated as `P * shift`, where the value of `shift` defaults to 0.001.
For all parameters that result in a continuous modification of the resulting cost function the variation of this cost function is going to be almost linear if vary small shifts are applied. As a consequence, the numerically derived derivative can be estimated just as well using just the two points at $P \pm h$ and you do not need all five points.
As REFINE does not know the significance behind a user supplied parameter the **newparam** command comes with the optional parameter **shift:** that lets you set the relative parameter

shift for each individual parameter. The optional parameter **points:** allows you to use 3 points or five points at which the cost function is estimated. Usually 3 points should be good enough. A special difficulty is encountered for refined parameters that cause a stepwise change in the simulated structure. The diameter of a (small) nanoparticle falls into this category. The number of atoms inside a small nanoparticle is a discrete integer number. Increasing the formally real valued diameter results in no structural change until the diameter is big enough to add one or more atoms. If such a parameter is modified by a small fraction or by a small absolute amount, no change in the structure might be encountered and the numerical derivative would appear to be zero. Only for a larger shift of the parameter will a significant structureal modification occur, which will in turn affect the R-value. A good value for diameters seems to be around 0.05, i.e. the current diameter is modified by `h= P*0.05`.
For disordered crystal structures the simulation will commonly involve the generation of random numbers. As an example take a small nanoparticle with stacking faults. A repeated simulation will create stacking faults acording to the user defined probbilities. Their exact location and the number of stacking faults will not be reproduced. Such a behavior, while intended for the main simulation, will likely cause impredictable results while REFINE attempts to estimate the derivative with respect to the stacking fault probability. To circumvent this erratic action, REFINE uses identical random number seeds for all those calculations that calculate the cost function during the determination of the derivatives.
A second aspect to consider while using a Least-Squares algorithm that is based on the numerical calculation of the derivatives is the presence of local minima.
A refinement within REFINE typically consists of two user supplied macro. In the main macro the refinement parameters, the input data and convergence criteria are defined. The actual calculation of the model function, respectively the model structure and its diffraction pattern, are carried out by the second macro.
The REFINE section uses four parameters to determine is the refinement has reached convergence. These are based on the absolute value of $\chi^2$, and on the change of $\chi^2$, the relative parameter change and a confidence level.
Convergence is considered to have been reached if either the absolute value of $\chi^2$ falls below a user defined leve, or if all three other criteria are met. These three criteria are:

- The value of $\chi^2$ changes less than a user defined value between two cycles
- The largest relative parameter shift is less than a user defined value. The relative parameter shift is defined as the abolute value of the parameter change between two cycles divided by the parameter uncertainty.
- The confidence level reaches a user defined level.

The correct values of $\chi^2$, of the parameter uncertainties and the confidence value depend on the correct values of the data uncertainties. Often the raw data do not provide an accurate estimate of the data uncertainties. In this case the best strategy to start with is to assign unit wheights to all data points. As this implies that the optimum $\chi^2$ and the confidence level will be unknown, REFINE also requires the user to set a maximum number of refinement cycles.
The REFINE section defines the parameters that shall be refined as DISCUS SUITE variable names. These variable names and their current values are passed on to the user macro that

is used to simulate the model structure. To tune the refinement, an initial value, a valid parameter range, and a status flag that indicates if this parameter is to be refined or to be kept fixed can be provided.

# Chapter 3

# Example refinements

## 3.1 Overview

A refinement macro within the REFINE section essentially consists of four steps:

- Loading data and experimental uncertainties
- Defining the parameters that are to be refined
- Defining details of the refinement control
- Running the actual refinement

At each cycle during the refinement, REFINE writes two macros. A macro called *refine_best.mac* stores the current best parameters and can be used to repeat the best calculation. The second macro *refine_new.res* is an updated copy of the current refinment macro. This allows you to continue an initial refinement, modify the scope and number of parameter or the model to be refined.

## 3.2 Simple noisy data function

In this first example the refinement to a simple function is illustrated. The data were calculated as a simple polynomial, see Fig. 3.1:

$$y(x) = P_{const} + P_{lin} * x + P_{quad} * x^2 + P_{trip} * x^3 \tag{3.1}$$

A Gaussian distributed random error was added to each data point with a sigma equal to the absolute y-value of each data point. If y was close to zero, the minimum sigma was set to 0.001. As y depends linearly on each of the parameters $P_i$, we can expect that this refinement will run smoothly. Essentially arbitrary starting values could be used to perform the refinement.
Two macro files are needed for the refinement, the main refinement macro and the macro that caculates the theoretical function. The main refinement macro that was used for this simple introductory example is:

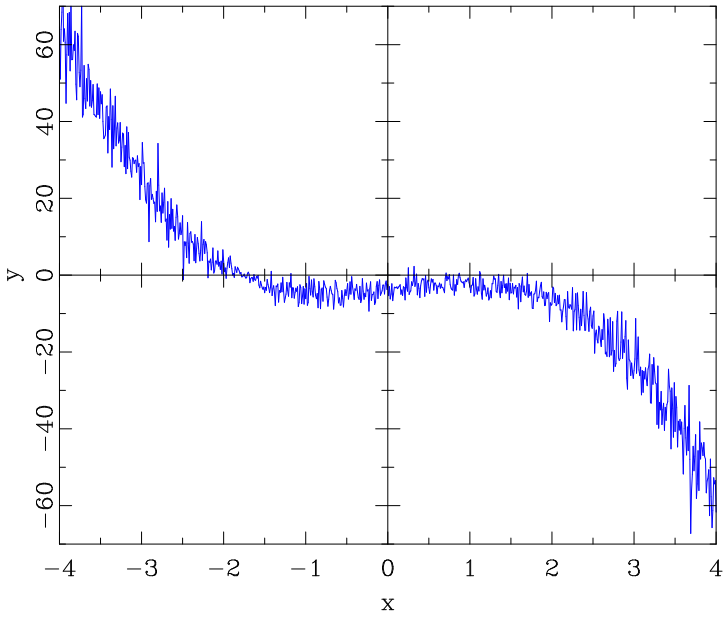



**Figure 3.1:** Experimental polynomial function

```
 0: refine
 1: reset
 2: data xy, DATA/triple.noise
 3: newparam, P_const, value:-8.01
 4: newparam, P_lin  , value:1.01
 5: newparam, P_quad , value:1.49
 6: newparam, P_trip , value:0.31
 7: set  cycle,   5
 8: set conv, dchi:0.50, pshift:0.005, conf:0.10, chisq:0.5
 9: set relax, start:0.02, success:0.5, fail:16.0
10: run triple.mac, plot:k_inter.mac
11: exit   ! back to SUITE
```

In line 0 we switch from the main DISCUS SUITE level to the REFINE section. As for all DISCUS SUITE menues, the `reset` command in line 1 puts REFINE into the startup condition. In REFINE this has a second effect, important for the macros `refine_best.mac` and refine_new.res that are produced after each cycle. To ensure that these macros will run, REFINE logs all non-refine commands that are encountered prior to `run` into the header of these two macros. Command `reset` clears this header! Make suer that all essential commands are placed after the initial `refine` command and between (an optional) `reset` command and the `run` command.
The data are loaded in line 2 as a simple "xy" file. All formats that are supported by the KUPLOT `load` command are available. REFINE stores the data set within its on memory. Thus no harm is done if KUPLOT resets the data sets during its calculations. During a refinement in which you need to calculate and average multiple powder or PDF data sets such a reset migh be the norm rather than the exception.
Lines 3 to 6 define the four parameters we wish to refine. For each parameter a suitable name needs to be defined. This name has to be a variable name that is valid within the DISCUS SUITE. If the variable name does not yet exist, it will be created. The command offers three optional parameters, only the "value" parameter is used in this example. It initiates the pa-

rameter to the user supplied value, which can be a numerical expression. As further optional parameter you can limit the range over which a parameter is valid by the parameter "range:[<lower>,<upper>]. This will be helpful if you need to exclude negative numbers or if the physical range of a parameter is limited. The last optional parameter allows to change the status of a parameter with "status:fix" or "status:fixed" the parameter will be fixed. If a "value:" parameter is given, the parameter will be fixed to this value, otherwise it takes its current value. If the optional parameter "status:" is omitted or if its value is set to "refine" or "free", its value will be refined.
Line 7 defines the maximum number of refinement cycles that the REFINE section will perform. If the convergence criteria are met in an earlier cycle, the refinement will stop at that point. Several criteria are used to determine if the refinement has reached convergence. In this example they are all specified on line 8. The four criteria are:

- chisq:<value> Convergence has been reached if the value of chi square drops below the user specified value.
- pshift:<value> AND conf:<value> AND dchi:<value> The largest parameter shift defined as the absolute value of the parameter shift divided by the abolute value of the parameter uncertainty AND If the confidence level is larger than this value AND The change in chi2 is less than this value then convergence is assumed.
- lambda:<max> Convergence is considerd to have been reached if lambda is larger than the specified maximum value.

If either the `chisq` or the `lambda` or all three criteria `conf`, `pshift` and `dchi` are met, convergence is reached and the refinement stops. Keep in mind that the value of chisq and the parameter uncertainty wil depend on the uncertainties of the input data values.
The Levenberg-Marquardt algorithm uses a parameter lamda to adjust the change of a parameter from one refinement cycle to the next. A small value results in a large parameter change and vice versa. REFINE will dynamically adjust this parameter during the refinement. Starting with a value of `0.02` the value of lambda is decreased by multiplication with `success:0.5` if the refinement succeeded i.e. chi2 decreased. The underlying assumption is that as long as the algorithm decreases chi2, we can take larger parameter steps. Once the algorithm does not find a better solution, its time to take smaller parameter steps in order to find the minimum, and lambda is increased with `fail:16.0`. The values in the xample are the default values. The main parameter you may have to play with is the start parameter. A larger value up to 2 seems to help difficult refinements.
The "run" command on line 10 will start the refinement using the macro "triple.mac" to calculate the theory data set. The optional parameter `plot:` allows to specify a user supplied macro that will display any data at each successful refinement step.

```
 1: branch kuplot
 2: #
 3: if(F_DATA==0) then
 4:    reset
 5:    func (P_const)+(P_lin)*r[0]+(P_quad)*r[0]**2+(P_trip)*r[0]**3, F_XMIN, F_XMAX, F_XSTP
 6: else
 7:    func (P_const)+(P_lin)*r[0]+(P_quad)*r[0]**2+(P_trip)*r[0]**3, F_DATA
 8: endif
 9: exit
10: finished
```

The refinement begins within the REFINE section of the DISCUS SUITE. Since we want to calculate a simple x-y data set, the KUPLOT section is the best choice. With the `branch kuplot` command in line 1 REFINE switches to KUPLOT. In line 3 the macro tests if REFINE loaded the data into a KUPLOT data set with data set number `F_DATA`. The REFINE build in variable `F_DATA` defines the data set number within KUPLOT into which REFINE was able to load the data set with the `data` command, line 2 in the main fit macro. The actual limits and the step width that REFINE detects are stored in the REFINE variables `F_XMIN`, `F_XMAX` and `F_XSTP`. Thes variables are all Read/Write but change their values with caution.
In KUPLOT the `func` command will calculate a user supplied function over a user defined range. The first parameter here the string
"(P_const)+(P_lin)*r[0]+(P_quad)*r[0]**2+(P_trip)*r[0]**3"
defines the function to be calculated. If three parameter follow, they define x-min, xmax and the x-step for the function. The DISCUS SUITE variable r[0] takes the role of the functions x-coordinate. If only one parameter is supplied, it defines the data set number whose range and step width are to be used for the function calculation. In the current example macro lines 5 and 7 will thus calculate the theory function over the identical range.
A reason to deveate from the limits of the data set occures when you refine against a powder diffraction data set. In order to take the effect of Bragg reflections slightly above the upper 2Theta limit into account, it is necessary to calculate the powder pattern to a larger 2Theta value. This will ensure that the low 2Theta tail of these Bragg reflections will be present within the calculated powder patttern, even if the peak position of the Bragg reflections is not.
The user macro needs to ensure that the last data set loaded into KUPLOT is the final theory curve.
At line 9 the user macro returns to REFINE . Finally in line 10, the REFINE command `finished` tells REFINE that the user macro is finished.
For the current macro the output produced during the refinement will be the following lines:

```
Cyc Chi^2/(N-P)   MAX(dP/sig)   Conf          Lambda       wRvalue      Rexp
  0   1.0222      0.26239      0.34233      0.50000E-03  0.25018      0.24745
  1   1.0219      0.14187E-02  0.34404      0.25000E-03  0.25015      0.24745

Convergence reached

 Information about the fit :
   Chi^2      :  814.484          Chi^2/N :  1.01683
   Conf. level: 0.344038      Chi^2/(N-P) :  1.02194
   No.Data    :          801     No.Params:           4
   MRQ final  : 0.250000E-03
   wR value   : 0.250150          R exp   : 0.247451

 Correlations larger than 0.8 :
   ** none **

   P_const          :   -3.92623      +-    0.107205
   P_lin            :    2.16236      +-    0.772162E-01
   P_quad           :   0.512809      +-    0.314855E-01
   P_trip           :   -1.11685      +-    0.134130E-01
```

The optional plot macro needs to start within the KUPLOT section and return with a final exit to REFINE. For the current example you could use:

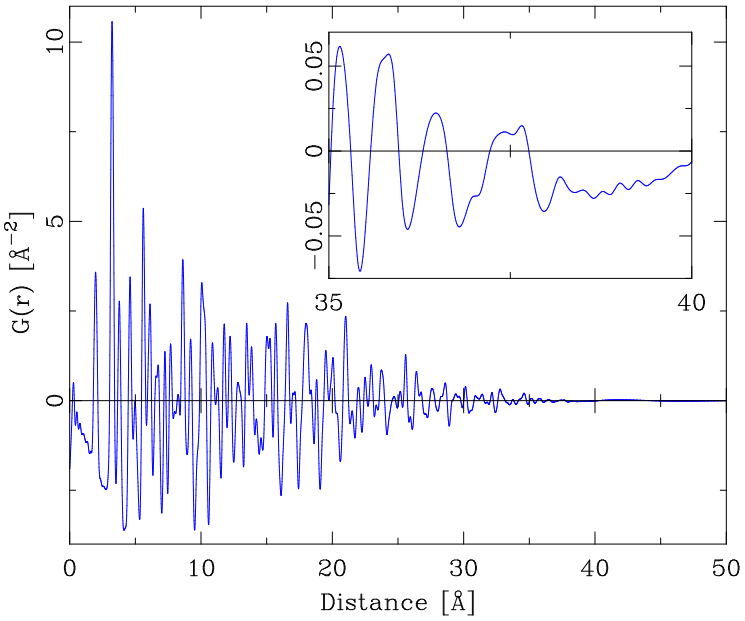



**Figure 3.2:** Experimental nanoparticle PDF. The nanoparticle has the perfect ZnO structure with a diameter of 45 Å. The insert shows enlarged the longer distances.

```
1: load xy, DATA/triple.noise
2: kccal sub, 2,1
3: r[0] = min(ymin[1],ymin[2]) - ymax[3]
4: ccal add, wy, 3, r[0]
5: skal
6: mark
7: rval 2,1, dat
8: reset
9: exit
```

At each successful refinement cycle, REFINE writes two new macros `refine_best.mac` and `refine_new.res`. Macro `refine_best.mac` is a short summary of the latest parameter values and has been designed such that you can reproduce the results in the last cycle. If the main refinement has been designed with temporary result files only, you could modify the working macros to produce final files on disk and create these at the DISCUS SUITE level with `refine_best.mac`.
Macro `refine_new.res` is a copy of your current refinement macro with updated parameter values. You can use this macro to continue a refinment with an improved or more complex model.

## 3.3 Nanoparticle PDF

In this section we will introduce the refinement of a nanoparticle PDF. This initial example is fairly straightforward. The data set is the calculated PDF of a spherical ZnO 45 Å diameter nanoparticle without any defects, Fig. 3.2. Thus we can, of course expect a rather perfect fit result.

The main parts of the main refinement macro are identical to the example in section 3.2.

```
 1: refine
 2: #
 3: data xy, DATA/zno.grobs
 4: newparam  P_lata   , value:3.220, range:[2.90,3.50], shift:0.0020 , points:5
 5: newparam  P_latc   , value:5.050, range:[4.70,5.30], shift:0.0020 , points:5
 6: newparam  P_z_zn   , value:0.360, range:[0.30,0.42], shift:0.0020 , points:5
 7: newparam  P_biso   , value:0.400, range:[0.00,]    , shift:0.0001 , points:5
 8: newparam  P_ab_dia , value:45.00, range:[20.0,80.0], shift:0.0100 , points:5
 9: #
10: newparam  P_eta    , value:0.600, range:[0.00,1.00], shift:0.0020 , points:5
11: newparam  P_eta_l  , value:0.000, status:fixed
12: newparam  P_u      , value:0.000, status:fixed
13: newparam  P_v      , value:0.000, status:fixed
14: newparam  P_w      , value:0.001, range:[0.00,0.10], shift:0.0020 , points:5
15: newparam  P_qmax   , value:24.00, status:fixed
16: newparam  P_scale  , value:1.000, range:[0.10,10.0], shift:0.0020 , points:5
17: set cycle,  25
18: set relax, start,2.00
19: set conver, dchi:0.5, pshift:0.005, conf:0.90, chisq:1.1
20: #
21: run discus_main.mac, plot:k_inter.mac
22: #
23: exit
```

As before, the data set is loaded as a simple x-y ASCII data set in line 3. The parameters are defined in lines 4 to 16. The main difference is the use of the additional optional parameters `range`, `shift` and `points`.

With `range` a range of allowed parameter limits can be defined. You can use this to restrict the refinement range to a sensible range, limited by intuition or by physical constraints. The optional parameter takes two values, the lower and upper boundary. One of these two values may be omitted, as in the case of `P_biso`. The lower limit is defined as 0.00, as a atomic displacement parameter cannot assume negative values. The upper limit is absent, allowing arbitrarily large values.

The next optional parameter `shift` allows to specify a multiplicative term that is used to calculate parameter points at `p-2*delta`, `p-1*delta`, `p-2*delta`, `p-2*delta`. These additional points are needed to calculate a numerical derivative of the function value with respect to the parameter at hand. An accurate derivative will be calculated at the current parameter value if the shifts `delta` are small. In a crystal structure simulation there are, however, a number of parameter types that need a rather large shift `delta`. The most prominent example of this class are nanoparticle diameters. If these are increased by a small amount, no atom might be added to the structure at all, as the boundary surface might cut in between the positions of all atoms. In the example a value of `shift:0.01` works well.

The last optional parameter `points` specifies how many points are used to calculated the derivative. Possible values are 3 or 5, which include the parameter value itself. The larger number of points will provide a more accurate derivative, but will of course require more computing time.

Another issue to be considered is the determination and transfer of fixed parameters from the main fit macro to the slave macro. Two different styles might be adopted. You can create variables and set their values within a separate macro, which you use at all appropriate macros. The other option, chosen in this example, is to use the `newparam` command but to fix the parameter value. It is mostly a matter of personal choice which style you prefer. As an argument

in favor of the `newparam` style, consider the profile parameters `P_u`, `P_v`. It will often be necessary to refine all three profile shape parameters. It makes sense though to start with the simplest model, a profile function of constant width, determined by parameter `P_w`. If, later on, you decide that a full profile function is needed, all you have to do is to free parameters `P_u` and `P_v`. For the style of a separate macro, you would have to remove the parameters from the variable defining macro, remove all initilization lines in any macro and add new parameters to the main fit routine. This is likely more error prone.
The main DISCUS macro DISCUS_main.mac has to build a spherical nanoparticle based on the current refinement parameters, calculate the necessary diffraction data and provide these as final data set within the KUPLOT section.

```
 1: branch discus
 2: variable integer, ncellx
 3: variable integer, ncellz
 4: #
 5: read
 6:   stru CELL/zno.cell
 7: lat[1] = P_lata
 8: lat[2] = P_lata
 9: lat[3] = P_latc
10: z[1]   = P_z_zn
11: b[1]   = P_biso
12: b[2]   = P_biso
13: ncellx = 2.00*int(P_ab_dia/lat[1]) + 2
14: ncellz = 2.00*int(P_ab_dia/lat[3]) + 2
15: save
16:   outf internal.zno.cell
17:   omit all
18:   run
19: exit
20: read
21:   cell internal.zno.cell     , ncellx, ncellx, ncellz
22: surface
23:   boundary sphere, P_ab_dia/2.
24: exit
25: purge
26: @powder.mac
27: @output.mac TEMP/zno.grcalc
28: branch kuplot
29: rese
30: load xy, TEMP/zno.grcalc
31: skal
32: ccal mul, wy, 1 , P_scale
33: ksav 1
34:   form xy
35:   outf   TEMP/zno.grcalc
36:   run
37: exit
38: exit
39: finished
```

In lines 5 to 12 a template asymmetric unit is read, its lattice parameters atom position and ADP values are adapted. The known lattice parameters are used to calculate the number of unit cells required to fit a spherical object into a block of unit cells in lines 13 and 14. The modified asymmetric unit is saved to an internal file (lines 15 to 19) and a block of unit cells is build using the current information, lines 20 and 21. Within the `surface` menu (lines 22 to 24) a sphere is cut with radius `P_dia/2` and all voids are removed from the structure, line 25. The

PDF is determined by calculating the powder diffraction pattern through the Debye Scattering Equation, macro `powder.mac` line 26. This PDF is saved with the `output` menu in line 27. In the remainder of the macro the PDF is scaled and its final form saved to the hard disk. The `exit` in line 37 returns to the DISCUS section and the `exit` in line 38 returns to the REFINE section. The slave macro is terminated by the special REFINE command `finished`.

```
 1: powder
 2: xray
 3: set axis,q
 4: set calc,debye
 5: set disp,off
 6: set delta,0.00
 7: set qmin, 0.5000
 8: set qmax, P_qmax
 9: set dq,   0.0001
10: set profile, pseudo
11: set profile, eta, P_eta, P_eta_l
12: set profile, uvw, P_u, P_v, P_w
13: set profile,  width, 11
14: set temp,use
15: set wvle,0.20000
16: set four,four
17: set lpcor,bragg,0.500
18: set scale, 1.000
19: run
20: exit
```

The powder menu uses the Debye Scattering Equation (line 4) to calculate the powder pattern for the finite sized spherical nanoparticle. While the lower limit $Q_{min}$ is fixed in this example to 0.5, the upper limit is determined by the (fixed) parameter `P_qmax`. A Pseudo Voigt profile function is set up in lines 11 to 13.

```
 1: output
 2: value PDF
 3: outf \$1
 4: form powder, r, 0.01,  50, 0.01
 5: run
 6: exit
```

In order to generate the PDF, the output menu is instructed to write the data with value `PDF`, line 2. Correspondingly, the format is set to `r` with user supplied limits in line 4. In this example, the limits are fixed number, if necessary this can be changed to use variable names. This refinement proceeds with the following initial output:

```
Cyc Chi^2/(N-P)   MAX(dP/sig) Par   Conf          Lambda       wRvalue      Rexp
  0   2273.9      0.31748E-02   5   0.0000      0.10000E-01   1.0180      0.21348E-01
  1   1476.8      0.15617       5   0.0000      0.50000E-02  0.82039      0.21348E-01
  2   1194.3      0.56799E-01   5   0.0000      0.25000E-02  0.73777      0.21348E-01
  3   862.37      0.59080E-02   6   0.0000      0.12500E-02  0.62690      0.21348E-01
  4   557.78      0.13358E-01   5   0.0000      0.62500E-03  0.50418      0.21348E-01
  5   329.67      0.14493E-01   5   0.0000      0.31250E-03  0.38761      0.21348E-01
  6   171.28      0.10872E-01   5   0.0000      0.15625E-03  0.27939      0.21348E-01
  7   138.25      0.17644E-01   6   0.0000      0.78125E-04  0.25101      0.21348E-01
  8   51.188      0.99726E-02   5   0.0000      0.39062E-04  0.15274      0.21348E-01
  9   26.416      0.29501E-01   6   0.0000      0.19531E-04  0.10972      0.21348E-01
 10   8.3212      0.51764E-01   6   0.0000      0.97656E-05 0.61581E-01   0.21348E-01
 11   6.7917      0.62021E-02   5   0.0000      0.48828E-05 0.55635E-01   0.21348E-01
 12   6.7917       0.0000       1   0.0000      0.78125E-04 0.55635E-01   0.21348E-01
```

```
 13   6.7917       0.0000       1   0.0000      0.12500E-02  0.55635E-01  0.21348E-01
 14   6.7917       0.0000       1   0.0000      0.20000E-01  0.55635E-01  0.21348E-01
 15   6.7917       0.0000       1   0.0000      0.32000      0.55635E-01  0.21348E-01
 16   5.8721      0.16346E-02   5   0.0000      0.16000      0.51731E-01  0.21348E-01
 17   4.7877      0.26307E-02   5   0.0000      0.80000E-01  0.46711E-01  0.21348E-01
 18   3.7652      0.35205E-02   5   0.0000      0.40000E-01  0.41424E-01  0.21348E-01
 19   2.8465      0.19826E-02   5   0.0000      0.20000E-01  0.36017E-01  0.21348E-01
 20   1.6043      0.16427E-02   5   0.0000      0.10000E-01  0.27039E-01  0.21348E-01
 21   1.2085      0.17484E-03   5  0.12624E-21  0.50000E-02  0.23468E-01  0.21348E-01
Convergence reached
```

As you can see the inital agreement is not very well, nevertheless the algorithm manages to find and refine into the perfect solution in cycle 21. Note that in cycles 12 to 15 the algorithm does not find a better solution and increases `lambda`. At cycle 21 the shift in chi squared is less than the user defined threshold and the refinement terminates.
The final output lists the convergence criteria, the final agreement values and the refined parameters.

```
 Data loaded as     : xy, DATA/zno.grobs
 Sigma not defined
 Refinement macro   : discus_main.mac
   Convergence 1    : dP/sigma < AND conf >     AND dChi^2 <
                      0.500E-002     0.900          0.500
   Convergence 2    : dChi^2 <   AND dP/sigma > 0.0
                      0.500
   Convergence 3    : Chi^2 <
                       1.10

 Information about the fit :
   Chi^2      :  6032.82          Chi^2/N :  1.20656
   Conf. level: 0.126237E-21  Chi^2/(N-P) :  1.20850
   No.Data    :         5000     No.Params:            8
   MRQ final  : 0.500000E-02
   wR value   : 0.234680E-01      R exp   : 0.213478E-01

 Correlations larger than 0.8 :
   P_w             - P_eta            : -0.953

 Refined parameters
   P_lata           :    3.24883      +-    0.126421E-04    [   2.90000     ,   3.50000     ]
   P_latc           :    5.20379      +-    0.296431E-04    [   4.70000     ,   5.30000     ]
   P_z_zn           :   0.367053      +-    0.108764E-04    [  0.300000     ,  0.420000     ]
   P_biso           :   0.462104      +-    0.327584E-03    [   0.00000     ,               ]
   P_ab_dia         :    39.7863      +-    0.129176E-01    [   20.0000     ,   80.0000     ]
   P_eta            :   0.867988      +-    0.174564E-01    [   0.00000     ,   1.00000     ]
   P_w              :   0.520064E-003 +-    0.197551E-04    [   0.00000     ,  0.100000     ]
   P_scale          :   0.986814      +-    0.837900E-03    [  0.100000     ,   10.0000     ]
 Fixed   parameters
   P_eta_l          :    0.00000
   P_u              :    0.00000
   P_v              :    0.00000
   P_qmax           :    24.0000
```

Fig 3.3 shows the excellent agreement between *observed* and calculated data, even at very large distances near the nanoparticle diameter.

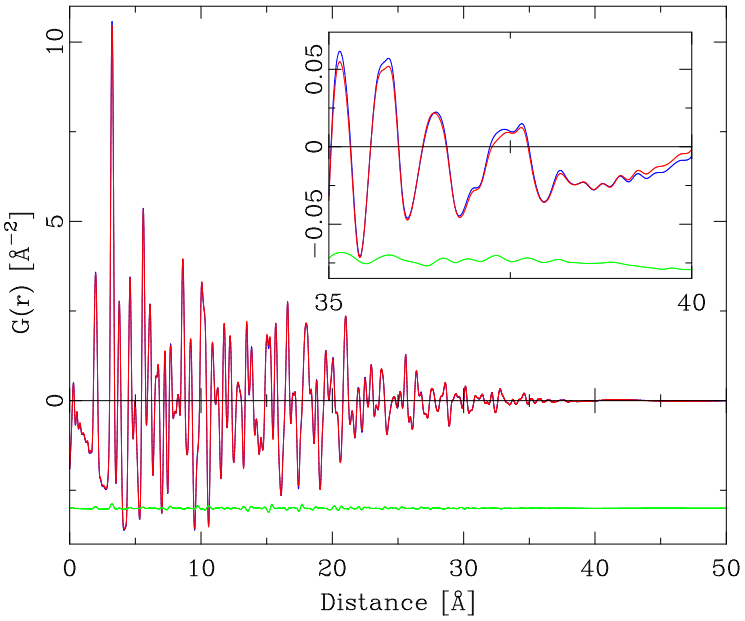



**Figure 3.3:** Experimental nanoparticle PDF overlayed with the final result. The difference curve in green is shifted down for clarity.

## 3.4 Single crystal refinement

The intent behind the DISCUS SUITE package is to simulate disordered crystal structures and to calculate the corresponding diffraction patter. Classical single crystal refinement is certainly not the main focus. Nevertheless, this may be helpful to ensure that the average crystal structure does agree well with the experimental Bragg reflection intensities.
In this section we demonstrate the refinement of a perfectly ordered crystal structure against calculated Bragg intensities. The Bragg intensities were calculated for a Gypsum crystal based on the following average structure.

```
title Gypsum from Boeyens  Ichharam Z.Krist 2002 217 9 in C2/c
spcgr C2/c, origin:1, setting:abc
cell         6.28400,      15.20000,       6.52300,      90.00000,     127.41000,      90.00000
scat  CA  , S   , O1  , O2  , OW  , H1  , H2
occ   1.00, 1.00, 1.00, 1.00, 1.00, 1.00, 1.00
adp   1.01, 2.01, 3.01, 4.01, 5.01, 6.01, 7.01
anis type:  1, values:[ 0.010500, 0.012400, 0.010400, 0.000000, 0.005700, 0.000000]
anis type:  2, values:[ 0.008400, 0.010800, 0.009000, 0.000000, 0.004600, 0.000000]
anis type:  3, values:[ 0.016300, 0.018700, 0.014500, 0.000600, 0.009300, 0.005200]
anis type:  4, values:[ 0.011500, 0.017900, 0.017200, 0.005200, 0.007500, 0.006800]
anis type:  5, values:[ 0.030400, 0.018200, 0.015700, 0.001300, 0.009900,-0.002800]
anis type:  6, values:[ 0.040000]
anis type:  7, values:[ 0.041000]
format numbers,XYZBPMMOS
atoms   X,       Y,        Z,     BISO, Property, MoleNo, MoleAt, Occ,St,  Sh,  Sk,  Sl
CA   0.500000, 0.079560, 0.750000,  1.010000,  1, 0,      0, 1.000000, _,   0,   0,   0
S    0.000000, 0.077240, 0.750000,  2.010000,  1, 0,      0, 1.000000, _,   0,   0,   0
O1   0.037200, 0.131980, 0.587200,  3.010000,  1, 0,      0, 1.000000, _,   0,   0,   0
O2   0.242400, 0.022110, 0.909200,  4.010000,  1, 0,      0, 1.000000, _,   0,   0,   0
OW   0.620200, 0.181970, 0.078400,  5.010000,  1, 0,      0, 1.000000, _,   0,   0,   0
H1   0.749000, 0.162000, 0.251000,  6.010000,  1, 0,      0, 1.000000, _,   0,   0,   0
```

```
H2   0.584000, 0.235000, 0.073000,  7.010000,  1, 0,      0, 1.000000, _,   0,   0,   0
```

The main simulation macro to calculate initial Bragg data is:

```
1: discus
2: #
3: read
4:   cell gypsum.stru
5: @fourier.mac gypsum_theo.hkl
6: read
7:   stru gypsum.stru
8: export shelx, gypsum_theo.ins, cycle:10
9: exit
```

The intensities are calculated in a block of reciprocal space in `fourier.mac` as:

```
 1: fourier
 2: #
 3: xray table:waas
 4: wvle MOA1
 5: temp use
 6: disp off
 7: #
 8: ll -8.0, -16.0, -8.0
 9: lr  8.0, -16.0, -8.0
10: ul -8.0,  16.0, -8.0
11: tl -8.0, -16.0,  8.0
12: na 17
13: no 33
14: nt 17
15: abs h
16: ord k
17: top l
18: set symmetry:ignore
19: set technique:turbo
20: set aver, 0.0
21: show
22: run
23: exit
24: output
25:   outf gypsum_theo.hkl
26:   value inte
27:   form hklf4, maxval:0001.000
28:   run
29: exit
```

This generates a block of intensities, and the resulting Bragg reflection list can in turn be used to refine the structur model. Often, the experimental list of Bragg reflection intensities will include multiple occurences of the same Bragg reflection `hkl`. Specialized single crystal refinement programs such as SHELX, OLEX, etc. will readily merge the multiple occurencies into a set of unique Bragg reflection intensities. As single crystal refinement is not the main purpose of the DISCUS SUITE , this merge operation is currently not included.
The REFINE section in particulare expects as input a list of Bragg reflection intensities which does not contain multiple occurences of a given Bragg reflection `hkl`. In order to demonstrate how to generate such a list of merge, unique Bragg reflections, the asymmetric unit is exported into a SHELX instruction file in line 8 of the main simulation macro. This instruction file is used in combination with SHELXL to run the single crystal refinement, and in particular to use the

SHELXL instruction`LIST 6` to write a list of merged reflections with `h,k,l, Fo-squared, sigma(Fo-squared), Fc and phi(calc).`
A single crystal refinement will often include a sizeable number of parameters that include 3 position parameters and 1 or six atomic displacement parameters for each atom in the asymmetric unit. Writing a main refinement macro that defines all these parameters and the corresponding DISCUS macro that simulates the structure and calculates the Bragg intensities can be a tedious and error prone process. To reduce the probability of errors, the DISCUS SUITE provides the optional parameter `refine:yes` to be used at the DISCUS command `import`, as in the following short macro:

```
discus
import shelx, gypsum_theo.ins, refine:yes
exit
```

If the optional parameter `refine:yes` is present, DISCUS will import the `SHELX` instruction file into a DISCUS structure file and at the same time create a refinement main macro, the DISCUS work macro and a small plotting macro to display the refinement qualtity. If the instruction file includes the `SHELX LIST` instruction the corresponding `.fcf` file will be read and converted into a `SHELX hklf 4` file type that will contain the set of merged unique reflection intensities.
For the current gypsum example, the resulting REFINE main macro is:

```
 1:  refine
 2:  rese
 3:  data hklf4, gypsum_merged.hkl
 4:  newpara P_scale, value:   1.0000, points:5, shift:0.01, status:free
 5:  newpara U_CA_1, value:0.10500000E-001, points:5, shift:0.030, status:free
 6:  newpara U_CA_2, value:0.12400000E-001, points:5, shift:0.030, status:free
...
37:  newpara P_CA_x, value:  0.50000000, points:5, shift:0.030, status:free
38:  newpara P_CA_y, value:  0.07956000, points:5, shift:0.030, status:free
39:  newpara P_CA_z, value:  0.75000000, points:5, shift:0.030, status:free
40:  newpara P_S_x, value:  0.00000000, points:5, shift:0.030, status:free
41:  newpara P_S_y, value:  0.07724000, points:5, shift:0.030, status:free
42:  newpara P_S_z, value:  0.75000000, points:5, shift:0.030, status:free
...
58:  newpara P_exti, value: 0.00000, points:5, shift:0.03, status:fixed
59:  set cycle,   5
60:  set conver, status:on, dchi:0.050, chisq:1.10, pshift:2.0, conf:1.0, lambda:65000.
61:  @gypsum_merged_main.mac
62:  branch kuplot
63:    @k_fobs_fcalc.mac
64:  run gypsum_merged_main.mac, plot:k_fobs_fcalc.mac
65:  exit  ! Back to SUITE
```

The hkl file name `gypsum_merged.hkl` indicates that the `fcf` file was used to create a set of unique, merged Brag reflection intensities. Following the overall scale factor, the isotropic / anisotropic atomic displacement parameters are defined, using their current values from the `SHELX` instruction file. Not all lines are reproduced. All atomic position parameter follow (lines 37 to 57). Note that in this current version atom positions like the Ca position at 1/2, y, 3/4 is given as freely refined parameter, although the x and z coordinates are fixed by symmetry. An extinction parameter is included in the refinement macro. If the `SHELX` instruction files includes the `EXTI` instruction the parameter is set with status:free, otherwise the parameter is fixed at 0.

Comparatively few refinement cycles are set in line 58 as the DISCUS SUITE expects the structure to have been refind within SHELX or another single crystal refinement program. Following the regular convergence criteria, the next three lines serve to calculate the set of Bragg reflections one using the staring parameters. This allows you to check if reasonable starting parameters were generated or if an unexpected error persists in the macros. Last, in line 64 the actual efinement is started.

# Chapter 4

# Creating a DISCUS SUITE refinement Macro Set

Writing all the required macros from scratch can easily be an error prone task. To ease this step the DISCUS SUITE includes an easy setup for a refinement template. The template can be build using a SHELX instruction file as the initial input. Such a file will provide a full set of information regarding a single crystal refinement, yet it can also be used to define at least a generic template for a powder diffraction and a Powder Pair Distribution PDF refinement in REFINE. As a SHELX instruction file does not contain any information regarding a diordered structure you will still have to do some editing of the macros that will have been generated with the setup procedure. You will, however, be able to start from an working example.
The DISCUS SUITE is not primarily designed to perform a classic single crystal or Rietveld type refiment. Specialized single crystal refinement programs like SHELXL respectively Fullprof, Topaz, Maud, GSAS-II etc. are certainly much better suited to perform this task. This setup is intended as a template to expand the initial refinement into a full refinement of a disordered structure.
In this introduction, the common set up scheme is explained. The individual sections that follow will focus on the individual details and in particular will explain which macros may have to be adapted for the refinement of a disordered structure.
All of the following sections assume that a SHELXL instruction file exists. If you do not have such a file at hand, DISCUS can generate the file with the instructions in this short macro.

```
1  discus
2  read
3  stru CELL/znse.cell
4  export shelxl, znse_wurtzite.ins, cycle:10, lambda:MOA1
```

The macro reads the asymmetric unit of the Wurtzite type modification of ZnSe and exports this information into a SHELXL instruction file. The optional parameter `cycle:10` instructs the instruction file to perform 10 refinement cycles. A further option is to provide the wave length as symbol or numeric value. Make sure that you read the asymmetric unit only.
All of the setups in the next four sections use a macro line within the DISCUS section equivalent to:

```
1  import shelx, znse_wurtzite.ins, names:shelx, refine:[style:single, form:waas, &
2         lambda:MOA1]
```

This command expects to find a file `znse_wurtzite.ins` in the current directory. The `import` command will read the SHELX instruction file and convert it into a discus `cell` file, here using the actual SHELX atom names.
The optional parameter `refine` and its parameters instructs DISCUS to set up a REFINE refinement scheme. See the DIFFEV manual for an equivalent set up for the DIFFEV section.
The `style` parameter defines what type of refinement should be created

| | |
|---|---|
| style:single | Run a single crystal refinement. DISCUS will look for an `hkl` or `fcf` file in the current directory. If either exists a Bragg reflection setup will be created, otherwise a 3D diffuse scattering setup. |
| style:powder | Run a powder diffraction refinement. |
| style:pdf | Run a PDF refinement. |

The `form` parameter lets you define the atomic form factors to be used:

| | |
|---|---|
| form:table | Use atomic form factors as tabulated in the International Tables Vol C |
| form:waas | Use atomic form factors as tabulated in Waasmeier and Kirfel. This is the default. |

The parameter `lambda` lets you define the wave length for the refinement. If absent, the wave length defaults to the value found in the SHELXL `CELL` instruction.
All styles will create a set of required directories:

| | |
|---|---|
| CELL | Will contain a copy of the cell file, the refinements read this copy. The macro at the beginning of this chapter read a cell file from the CELL folder. Such a cell file will be replaced by the result of the `import` command! |
| DATA | Place the observed data into this directory. An old directory is not changed. |

All styles generate a macro `refine_SUBSTANCE_style.mac` that needs to be used as the main refinement macro. Here and in all following macro lines the string tt SUBSTANCE should be understood as being replaced by the name of the SHELXL instruction file, in these examples it would be `znse_wurtzite`. the string `STYLE` will be one of `dat; merged; powder; pdf` depending on the chosen style. This main refinement macro should be started at the top DISCUS SUITE level.
To test the initial macro, especially when you need to add code to perform a simulation of a disordered structure the setup writes the main macro such that the main DISCUS work macro and a plot macro are invoked prior to the actual refinement. If you need to continue development, comment the `run` command that follows.
The main macro will perform only 20 cycles and use a very restricted set of refined parameters. You will need to adjust these to your individual need. REFINE defines the refined parameter names as `P_the_name` and `U_Atom_N` for the anisotropic atomic displacement parameters. These parameter names are not fixed, feel free to adjust to your own style. During the setup DISCUS checks the SHELXL instruction file and defines as parameters only any atom coordinate that is not fixed by symmetry.
As an example the following macro is generated for single crystal Bragg intensity refinements:

```
1 refine
2 rese
3 data hklf4, znse_wurtzite_merged.hkl
4 newpara P_scale, value:   1.0000, points:3, shift:0.001, status:free
5 newpara U_ZN_1, value:0.58500000E-002, points:3, shift:0.001, status:free
```

```
 6 newpara U_SE_1, value:0.58500000E-002, points:3, shift:0.001, status:free
 7 newpara P_ZN_z, value:  0.36710000, points:3, shift:0.001, status:free
 8 newpara P_SE_z, value:  0.00000000, points:3, shift:0.001, status:free
 9 newpara P_exti, value: 0.00000, points:3, shift:0.001, status:fixed
10 set cycle,    20
11 set conver, status:on, dchi:0.050, chisq:1.10, pshift:2.0, conf:1.0, lambda:65000.
12 @znse_wurtzite_main.mac
13 branch kuplot
14   @k_fobs_fcalc.mac
15 run znse_wurtzite_main.mac, plot:k_fobs_fcalc.mac
16 #
17 @final_cell_cif.mac
18 #
19 exit  ! Back to SUITE0
```

This macro is very similar to the initial macro in chapter 3. Three aspects need to be mentioned here.
The `data` command in line 3 will load the data file from folder DATA. DISCUS does an educated guess to the file format and name to be expected in this directory, adjust as needed. REFINE reads the file with the intention to populate variables `F_XMIN, F_XMAX, F_XSTP` and equivalently Y and Z for 3D data. These limits are subsequently used in the macros that calculate / write the calculated data. Furthermore, this data set is stored internally and used to compare its values to the calculated pattern.
The automatic setup generates an educated guess for all required parameters and initially restricts many of these to fixed values. See the individual sections for further information on the parameter list.
As a SHELX instruction file assumes xray diffraction you will have to edit the `fourier` or `powder` menus in the resulting macros to switch to neutron or electron diffraction.

## 4.1 Setting up a single crystal Bragg refinement

With `style:single` DISCUS will look for files `SUBSTANCE.hkl` and `SUBSTANCE.fcf`. If either is found, a single crystal refinement using the integrated Bragg intensities in these files will be created. As the `fcf` file contains a merged set of the Bragg reflection intensities the refinement will be faster. Without modifications the refinement will replicate a SHELXL single crystal refinement. If the `fcf` file is present, DISCUS converts this into an hklf4 format under the name `SUBSTANCE_merged.hkl`
In the initial setup most atom positions and thermal parameters and the scale factor `P_scale` are free to refine. Check if this gives a stable refinement.
The `data` command in `diffev_main_SUBSTANCE.mac` will load the hkl file with the hklf4 type.

## 4.2 Setting up a single crystal refinement

If the current directory does not contain an `hkl` or `fcf` file, DISCUS will assume that a 3D diffuse scattering calculation is intended. You need to make sure that the DATA directory contains a suitable file. If the observed data are not in the hdf5 file format, three macro files:

`refine_SUBSTANCE_STYLE.mac; k_inter.mac; SUBSTANCE_main.mac`
need to be modified.
In the main DISCUS macro `SUBSTANCE_main.mac` check the commented lines

```
1 !set filter, lanczos, damp:0.5, width:7, scale:1.0
2 !set symmetry, apply
3 run ! sigabs:[0.001, 1.0,0.0,0.0, sigord:[0.001, 0.0,1.0,0.0], sigtop:[0.001, 0.0,0.0,1.0]
```

Do you need to smooth the data with a filter (line 1), is symmetry averaging allowed (line 2) and does it help your calculated data. Lastly do you need to convolve the experimental data with a resolution function (comments on the run command, line 3).
Check if the `nufft` technique works fine for your sample.
Another likely modification is to switch the calculation to a $3D$-$\Delta$-$PDF$. Besides the observed data you will need to modify the output in the main DISCUS macro `SUBSTANCE_main.mac` to something like:

```
 1 fourier
 2   ...
 3   set aver, 100
 4   ...
 5   run
 6 exit
 7 output
 8   outf calc.h5
 9   form hdf5
10   value 3DPDF
11   run
12 exit
```

With line 3 `set aver, 100` a total of 100 percent of the actual crystal is sampled to calculate the Bragg intensities, which are subtracted from the calculated diffraction pattern. This creates the $3D$-$\Delta$-$PDF$ instead of a 3D-PDF.
A $3D$-$\Delta$-$PDF$ makes sense for an extended simulated crystal with disorder only. You will have to expand the crystal to a larger block of unit cells in the main DISCUS macro where the initial internally saved asymmetric unit is expanded at line `cell internal.SUBSTANCE.cell` and introduce the required disorder.

## 4.3 Setting up a powder diffraction refinement

With `style:powder` DISCUS will create a REFINE refinement for powder diffraction. The variable set and the powder macro will contain fixed parameters for the profile function and comments to assist in the setup for preferred orientation. Initially the scale parameter and the lattice parameters are free, all atom positions, thermal parameters and a $2\Theta$ zero point are fixed. The `data` command and the output in the main substance macro assume a $2\Theta$ scale.
In macro `kup.diffev.mac` multiple individual repetitions (if requested) are merged and a background polynomial is fitted. The sum of the background and the scaled calculated powder pattern is saved and serves as the total calculated data set.

## 4.4 Setting up a PDF refinement

With `style:pdf` DISCUS will create a REFINE refinement for the powder pair distribution PDF. The generated macros are mostly identical to the powder case. The main difference is that the automatic setup assumes an ellipsoidally shaped nanoparticle. Accordingly the powder pattern is calculated via the Debye-scattering-equation (DSE) assuming a wave length of 0.2Å and $Q_{max} = 26$Å. As the SHELXL instruction file does not contain any information on a nanoparticle shape or size, dummy values for the three diameters are used. These need to be adjusted for your own data.
The `data` command assumes that the observed PDF is present as a four column file with columns `r PDF 0 sigma(PDF)` with equidistant r scale. Adjust if needed. The limits and step sizes are copied into the calculated PDF.

## 4.5 Adding disorder to a refinement

Disorder within a simulated crystal is very much an individual problem. Consult the examples and algorithems in the DISCUS manual and the Cook book for general guidelines. The initial Nanoparticle example in the DIFFEV example chapter details the simulation of a finite sized nanoparticle with stacking faults.
For a single crystal refinement (diffuse or $3D$-$\Delta$-$PDF$) keep the simulated crystal to a block shaped object, see the notes on the crystal size in section 4.2. This facilitates periodic boundary conditions and enables a fast internal lookup that is often needed for example within the Monte-Carlo-menu `mmc`.
An important design choice is the question whether the disorder is represented within a single simulated crystal or if many individual crystals are required to obtain an ensemble average that corresponds to the experimental situation. Often, a suitably large single crystal will be a good represenation, especially if the correlations between the structural defects do not extend over many unit cells. A good example of the latter is the first example in the DIFFEV example chapter. A single very small ZnSe cannot represent the multitude of nanoparticles with different number and location of the stacking faults. Many particles must be simulated and their averaged powder diffraction pattern or PDF will be a good represenation of the actual experimental situation.
If no individual repetitions are needed the macros created by the automatic setup are good to use. Ottherwise you will have to add a loop over all required individual repetions with the main DISCUS macro. Make sure you average all these data in a KUPLOT section and load the averaged data as last data set before returning to the main REFINE part.

# Chapter 5

# Mathematical Background

## 5.1 Least squares algorithm

This chapter describes the mathematical background to the least squares algorithm implemented in REFINE. The description largely follows the discussion in the Numerical recipes Price et al. (2005).
A least squares algorithms works on the basic assumption that the we are able to describe our data by a model function that depends on M parameters:

$$y \; = \; F(x; p_0, p_1, ..., p_M) \tag{5.1}$$

If the uncertainties of the data are subject to a Gaussian distribution, the highest probability for any model parameters is given if the weighted residual. The weights are the inverse of the variance (= squared uncertainties) of the data points. $w_i = 1/\sigma_i^2$:

$$wR = \sqrt{\frac{\sum_i w_i (y_{obs}(i) - y_{calc}(i))^2}{\sum_i w_i y_{obs}(i)^2}} \tag{5.2}$$

is at its lowest possible value. In this equation for the weighted residual the sums run over all observed data points `i`. These data points might be the individual data points in a powder pattern, in a PDF or a two or three dimensional data set. The data points may likewise include several data sets and possible the need for special functions for a subsection of the data set. This latter situation occurs if we perform a simultaneous refinement of a structural model aginst a neutron and X-ray data set. Note that there is no need for the data points to be arranged in any special sequence of `i`.
Often the weighted residual as defined in Eq. 5.2 is replaced by the term "chi-squared", defined as:

$$\chi^2 = \sum_i w_i (y_{obs}(i) - y_{calc}(i))^2 = \sum_i \frac{(y_{obs}(i) - y_{calc}(i))^2}{\sigma_i^2} \tag{5.3}$$

A necessary condition for a minimum of the residual is that all derivatives of `wR` with respect to any parameter is zero. This condition holds of course for the squared residual as well:

$$wR^2 = \frac{\sum_i w_i (y_{obs}(i) - y_{calc}(i))^2}{\sum_i w_i y_{obs}(i)^2} \tag{5.4}$$

Thus, for each parameter j we have the partial derivative:

$$\frac{\partial wR^2}{\partial P_j} = \frac{2}{\sum_i w_i y_{obs}(i)^2} \sum_i w_i \left(y_{obs}(i) - y_{calc}(i)\right) \frac{\partial (y_{obs}(i) - y_{calc}(i))}{\partial P_j} = 0 \tag{5.5}$$

Since the observed values are independent of the parameters the equation simplifies to:

$$\frac{\partial wR^2}{\partial P_j} = -\sum_i w_i \left(y_{obs}(i) - y_{calc}(i)\right) \frac{\partial y_{calc}(i)}{\partial P_j} = 0 \tag{5.6}$$

To simplify the notation we will replace $y_{obs}(i)$ by $y_i$ and $y_{calc}(i)$ by $f_i$. This change in notation has our equations for the partial derivatives read:

$$\frac{\partial wR^2}{\partial P_j} = -\sum_i w_i \left(y_i - f_i\right) \frac{\partial f_i}{\partial P_j} = 0 \tag{5.7}$$

A total of M such equations must be solved for all parameters P from 1 to M.

### 5.1.1 Linear least squares algorithm

A reasonable straightforward solution to find the optimum parameters $P_j$ exists if the function $f_i$ can be written as a linear combination of the parameters:

$$f_i = \sum_j P_j g_{ij} \tag{5.8}$$

Here $g_{ij}$ can be any function of the data points `i`. As an example of such a linear combination take any polynomial function:

$$y_i = P_0 + P_1 * x_i^1 + P_2 * x_i^2 + P_3 * x_i^3 + ... \tag{5.9}$$

Here the functions $g_{ij}$ are powers of x at point `i`. We are now ready to substitute $f_i$ and its derivative with respect to parameter `j` in Eq. 5.7 by the function definition in 5.8 and obtain:

$$0 = -\sum_{i=1}^{N} w_i \left( y_i - \sum_{k=1}^{M} P_k g_{ik} \right) g_{ij} \tag{5.10}$$

This equation must hold for all parameters j=1 to M, and we can write all these equations as:

$$\sum_{k=1}^{M} \sum_{i=1}^{N} w_i \, g_{ik} \, g_{ij} P_k = \sum_{i=1}^{N} w_i \, y_i \, g_{ji} \qquad \text{j=1 to M} \tag{5.11}$$

This set of equations can be written concisely if we introduce a few matrices. The first is the so called design matrix A. Its elements are the values of the theory functions $g_{ij}$ divided by the experimental uncertainties at data point `i`, with $\sigma_i = \sqrt{w_i}$:

$$\mathbf{A} = \begin{pmatrix} \frac{g_{11}}{\sigma_1} & \frac{g_{12}}{\sigma_1} & \cdots & \frac{g_{1M}}{\sigma_1} \\ \frac{g_{21}}{\sigma_2} & \frac{g_{22}}{\sigma_2} & \cdots & \frac{g_{2M}}{\sigma_2} \\ \vdots & & & \\ \frac{g_{N1}}{\sigma_N} & \frac{g_{N2}}{\sigma_N} & \cdots & \frac{g_{NM}}{\sigma_N} \end{pmatrix} \tag{5.12}$$

Each row corresponds to the M functions $g_{ij}$ evaluated for data point `i`, while each column corresponds to a specific function evaluated at all data points. Like wise we can group the parameters $P_k$ into a column matrix:

$$\mathbf{a} = \begin{pmatrix} P_1 \\ P_2 \\ \vdots \\ P_M \end{pmatrix} \tag{5.13}$$

Next we define a column matrix for the N observed data points as :

$$\mathbf{b} = \begin{pmatrix} \frac{y_1}{\sigma_1} \\ \frac{y_2}{\sigma_2} \\ \vdots \\ \frac{y_N}{\sigma_N} \end{pmatrix} \tag{5.14}$$

This allows to write set M set of Eqs. 5.11 as:

$$\left(\mathbf{A}^T\mathbf{A}\right)\mathbf{P} = \mathbf{A}^T\mathbf{b} \tag{5.15}$$

Since $\mathbf{A}^T\mathbf{A}$ is a square MxM matrix, we can solve the equation to obtain the parameters as:

$$\mathbf{P} = \left(\mathbf{A}^T\mathbf{A}\right)^{-1}\mathbf{A}^T\mathbf{b} \tag{5.16}$$

The inverse matrix $\left(\mathbf{A}^T\mathbf{A}\right)^{-1}$ is called the *covariance matrix* **C**. Its elements allow to estimate the uncertainties of the parameters $P_k$ and any correlations between two parameters. Each element of the column matrix P in equation 5.16 is:

$$P_k = \sum_{j=1}^{M} C_{kj}\left(\sum_{i=1}^{N}\frac{y_i g_{ij}}{\sigma_i^2}\right) \tag{5.17}$$

The variance (= squared uncertainty) of parameter k can be calculated as:

$$\sigma^2(P_k) = \sum_{i=1}^{N}\sigma_i^2\left(\frac{\partial P_k}{\partial y_i}\right)^2 \tag{5.18}$$

Note that in Eq. 5.17 the elements $g_{ij}$ are independent of $y_i$. Furthermore in the sum over `i` the derivative with respect to $y_i$ is non-zero for element `i` only. Thus the derivative in Eq. 5.18 is:

$$\frac{\partial P_k}{\partial y_i} = \sum_{j=1}^{M} C_{kj}\frac{g_{ij}}{\sigma_i^2} \tag{5.19}$$

We can insert this derivative in squared form into Eq. 5.18, interchange the sequence of the sums to write:

$$\sigma^2(P_k) = \sum_{j=1}^{M}\sum_{l=1}^{M} C_{kj}C_{jl}\left(\sum_{i=1}^{N}\frac{g_{ij}g_{il}}{\sigma_i^2}\right) \tag{5.20}$$

The right hand term corresponds to the elements of $\left(\mathbf{A}^T\mathbf{A}\right)$, which in turn is the invariant matrix to $\mathbf{C}$. Thus the equation reduces to:

$$\sigma^2(P_k) = C_{kk} \tag{5.21}$$

The diagonal element of matrix $\mathbf{C}$ are the variances of the parameters k.

### 5.1.2 Non-linear least squares algorithm

Most of the times, the function $f_i$ cannot be written as a linear combination of the parameters. As an example take a function like:

$$f_i = P_1\cos(P_2 \cdot x_i) \tag{5.22}$$

Under these circumstances, the individual parameters cannot be separated from each other. Iterative strategies have been developed to find the minimum. Independend of the complexity of the function $f_i$, in close proximity to the minimum one can expect to approximate the value of $\chi^2$ as a quadriatic function of the parameters $P_i$:

$$\chi^2(P_j) = const - d_jP_j + \frac{1}{2}P_jD_{jk}P_k \tag{5.23}$$

Here, we can write this equation in more concise form if we use a matrix formalism. With matrix **a** from Eq. 5.13, a vector of length M (= number of parameters), and an MxM matrix $\mathbf{D}$:

$$\chi^2(a) = const - \mathbf{d}\,\mathbf{a} + \frac{1}{2}\mathbf{a}^T\mathbf{D}\mathbf{a} \tag{5.24}$$

Very close to the minimum, we can hope for the approximation to be a good one which allows us to find the best parameters with a single step:

$$\mathbf{a}_{best} = \mathbf{a}_{current} + \mathbf{D}^{-1}\cdot\left(-\nabla\chi^2(\mathbf{a}_{current})\right) \tag{5.25}$$

Unfortunately, most of the time we cannnot know how good our current approximation will be. The best one can do is to calculate the gradient of the squared residual and take a small step into the direction of the steepest decent. At this new point in the parameter space one needs to calculate the gradient again and take further steps towards the minimum. At any iteration during this procedure, the new parameter set can be calculated as:

$$\mathbf{a}_{next} = \mathbf{a}_{current} - constant \times \nabla\chi^2(\mathbf{a}_{current}) \tag{5.26}$$

The tricky part is the determination of the value of *constant*. It must be small enough to take a step downhill, and not beyond the minimum. A very small value will, however, increase the number of iterations it takes to reach the minimum. A further risk with a very small value of

the *constant* is to step into a local minimum. Thus different values of the *constant* and a dynamic adapration are needed.
As for the linear model we have out function:

$$y \;=\; F(x; p_0, p_1, ..., p_M) \tag{5.27}$$

and we need to minimize chi squared:

$$\chi^2 = \sum_i w_i (y_{obs}(i) - y_{calc}(i))^2 = \sum_i \frac{(y_{obs}(i) - y_{calc}(i))^2}{\sigma_i^2} \tag{5.28}$$

As in the introductory part we will reduce the terminology to:

$$\chi^2 = \sum_i w_i (y_i - f_i)^2 \tag{5.29}$$

In order to take steps towards the minimum we need to calculate the gradient of $\chi^2$ with respect to each parameter k:

$$\frac{\partial \chi^2}{\partial P_k} = -2 \sum_{i=1}^{N} \frac{(y_i - f_i)}{\sigma_i^2} \frac{\partial f_i}{\partial P_k} \tag{5.30}$$

A further derivative with respect to another parameter give the cross terms needed to a direct decent:

$$\frac{\partial^2 \chi^2}{\partial P_k \partial P_l} = 2 \sum_{i=1}^{N} \frac{1}{\sigma_i^2} \left( \frac{\partial f_i}{\partial P_k} \frac{\partial f_i}{\partial P_l} - (y_i - f_i) \frac{\partial^2 f_i}{\partial P_k \partial P_l} \right) \tag{5.31}$$

The individual equations 5.31 can be considered to be matrix elements of a matrix $\alpha$ with:

$$\alpha_{kl} \;=\; \frac{1}{2} \frac{\partial^2 \chi^2}{\partial P_k \partial P_l} \tag{5.32}$$

and a vector $\beta$:

$$\beta_k \;=\; -\frac{1}{2} \frac{\partial^2 \chi^2}{\partial P_k} \tag{5.33}$$

Here the factors $1/2$ have been introduced for convenience sake. With these definitions the parameter shift can be written as a vector $\delta$ and the equation to be solved becomes:

$$\mathbf{A}\delta \;=\; \beta \tag{5.34}$$

In the algorithm implemented in REFINE, as taken from Price et al. (2005) the second derivatives in Eq. 5.31 is omitted, making the matrix elements $\alpha_{kl}$:

$$\alpha_{kl} \;=\; \sum_{i=1}^{N} \frac{1}{\sigma_i^2} \left( \frac{\partial f_i}{\partial P_k} \frac{\partial f_i}{\partial P_l} \right) \tag{5.35}$$

The reasoning behind this omission is that the second derivatives tend to be much smaller, tend to destabilize the algorithm due to the noise in the experimental data and the final parameter values do not depend on the second derivatives, just the path towards the final parameters.

In terms of the matrices introduced in Eqs. 5.32 and 5.33, the steepest decent in Eq. 5.26 corresponds to

$$\delta a_l = constant \times \beta_l \tag{5.36}$$

Far from the minimum, this decent is the optimum method, while close to the minimum a solution according to 5.34 serves best. The commonly used Levenberg-Marquardt method provides an algorithm to vary between these two algorithms. The algorithm is described by Marquardt (1963) based on earlier suggestions by Levenberg (1944).
There are two parts to the Levenberg-Marquardt algorithm. First of all, a scale factor must be included into equation 5.36 to convert the dimensions of $\beta$ into those of the parameters $\alpha$. This conversion factor will likely be different for each parameter $\alpha$. The diagonal elements of matrix **A** in Eq. 5.32 do provide this vonversion factor. The Levenberg-Marquadrt algorithm provides an additional dimensionless factor $\lambda$ to scale the parameter shift, turning Eq. 5.36 into

$$\delta a_l = \frac{1}{\lambda \alpha_{ii}} \times \beta_l \tag{5.37}$$

The second part to the Levenberg-Marquardt algorithm provides a transition between the steepest decent and the direct solution. The matrix $\alpha$ is replaced by a new matrix $\alpha'$, where the diagonal elements are replaced according to:

$$\alpha'_{jj} = \alpha_{jj}\ (1\ +\ \lambda) \tag{5.38}$$

while the off diagonal elements remain the same.
The steepest decent, Eq. 5.36, and the direct solution close to the minimum, Eq. 5.34, are combined into a single new equation:

$$\boldsymbol{\alpha'} \delta \boldsymbol{a} = \boldsymbol{\beta} \tag{5.39}$$

The implementation of the Levenberg-Marquardt algorithm used in REFINE is taken from the numerical recipes Press et al. (1989) and is initialized by the following steps:

- Provide initial guesses for all parameters **a**.
- Provide initial guesses for $\lambda$.
- Calculate the corresponding value of $\chi^2$.

The program then carries out the iterations by:

- Solving Eq. 5.39 for the parameter shifts.
- Calculate $\chi^2$ for the modified parameters $a + \delta a$.
- If the iteration succeeds i.e. the new $\chi^2$ is less than the previous one, the new parameter values are accepted and $\lambda$ is decreased by a suitable factor.
- If the iteration did not succedd, i.e. the new $\chi^2$ is higher than the previous one, the old parameter set is retained and $\lambda$ is increased by a suitable factor.

REFINE enables the user to provide a starting value for $\lambda$ and individual factors for the decrease and increase.

# Appendix A

# REFINE commands

## A.1 Summary

Here is a list and brief description of valid REFINE commands. Further help can be obtained by typing the corresponding command name at the help prompt.

```
data     ! Defines the data set
finished ! Terminates the slave macro
fix      ! Fixes a parameter, free with newparam
newparam ! Defines and sets a new parameter, which is to be refined
reset    ! Performs a global reset to system start conditions
run      ! Starts the refinement
show     ! Shows settings
set      ! Sets control values like cycles, convergence criteria
sigma    ! For 2D data; defines a file that contains experimental uncertainties
```

## A.2 News : Information on program updates

This is the initial release of the REFINE section.

### 2020_Dec

Added variables par und sig that are read only

## A.3 Example

A simple fit might refine parameters of a straight line to observed data. Lets assume that the data are in a simple x/y file "observed.data". A suitable macro to refine is:

```
refine                      ! Switch from SUITE to REFINE section
data xy, observed.data      ! Load data set "observed.data"
                            ! The load command creates user variables
                            ! F_DATA = Number of Data set in Kuplot
                            ! F_XMIN, F_XMAX, F_XSTP
                            ! F_YMIN, F_YMAX, F_YSTP
                            ! That contain the data limits and step size.
```

```
newparam P_inter, value:1.0  ! Define y-axis intersept as first parameter
newparam P_slope, value:1.0  ! Define slope as second parameter
set cycle, 10                ! Define maximum number of cycles
run fit_work.mac             ! Start the fit with user macro fit_work.mac
exit                         ! Back to SUITE
```

The user macro fit_run should be something like:

```
branch kuplot                ! Step into the KUPLOT section
                             !   or for structures into DISCUS
func P_inter + P_slope*r[0], F_DATA
                             ! Calculate the function,
                             !   limits are set automatically from
                             !   data set F_data that was loaded by refine.
exit                         ! Back to REFINE
finished                     ! Special keyword signals end of user macro
```

## A.4 manual

```
manual ["section:"{"suite"  | "discus"  | "diffev" |
                   "kuplot" | "package" | "mixscat"}
       [,"viewer:"<name>]
```

Opens a PDF viewer for one of the Manuals
The section defaults to the current program section that you are working with. On Linux systems, the viewer defaults to "qpdfview", on Windows system it defaults to "firefox". If DISCUS does not find the default or the user provided viewer, DISCUS will search a list of common PDF viewers. If none is found an error message points to the folder that contains the manuals.

## A.5 data

```
data "kuplot", <number>
data <type>, <infile>
data "kuplot", <number>
```

The calculation will refine parameters versus the observed data that are stored in the KUPLOT data set number <number>

```
data <type>,<infile>
```

The calculation will refine parameters versus the observed data that are loaded from file <infile>.
See the ==> 'kuplot/load' command for details on proper ways to load a data set.
Refer to the KUPLOT section for instructions on loading data.

## A.6 finished

```
finished
```

This command must be the last instruction in any slave macro. The command instructs refine to terminate execution of the slave macro. REFINE will then evaluate the results and proceed to the next refinement cycle.

## A.7 fix

```
fix <parname> [,"value:"<number>]
```

Fixes a parameter. Its value will remain at its current value or at <number>, if the optional parameter "value:" is used.
A fixed parameter can be freed with a ==> 'newparam' command.

## A.8 newparam

```
newparam <parname> [,"value:"<start>] [,"status:"<flag>]
                   [,"range:["<lower>,<upper>"]"]
                   [,"shift:"<value>] [,"points:"<nderiv>]
```

The command performs three tasks: - Defines a new parameter named <parname> - Frees a parameter <pname> that had previously been fixed by the ==> 'fix' command - Defines a constant <pname> that you may need to use in the user macro.
Defines a new parameter name <parname>. This should be any valid user defined variable name, limited to a length of 16 characters. The user variable is allowed to have been defined previously, and its current value will be used if the "value:" option is omitted.
See the general command line section for details on the definition of variables.
Optional parameters are: value:<start> The Parameter will be initialized to the <start> value. If omitted, the parameter value will take on its current value.
status:refine status:free status:fix status:fixed With the flags "refine" or "free" the parameter will be refined during the refinement cycles. With the flags "fixed" or "fix", the parameter will remain fixed at its current value. Be careful that the user macro does not change the parameter value! Default flag is "refine".
range:[<lower>, upper>] Defines a lower and upper boundary for the parameter. The fit will ensure that the parameter does not move outside the specified range. <lower> must be less than <upper>. If either <lower> or <upper> is absent, but the comma is present: [,<upper>] or [<lower>,] the missing side is not constrained. If the "range:" parameter is omitted, the default behavior is to assume no boundaries. In order to turn the boundaries off, simply state the 'newparam' command again for the refinement parameter without the "range:" option.
shift:<value> To calculate the numerical derivatives REFINE needs to calculate the cost function at the current parameter value and at points at slightly smaller and slightly larger values. REFINE calculates the shift of the original parameter values as par*<value>. It is not straightforward to recommend good values. For parameters that can be expected to give a smooth variation of the cost function like scale factors a value of 0.003 seems to be good. For parameters like nano particle diameters larger values like 0.1 to 0.01 seem to be better. Default: shift:0.003 For parameters called "P_dia" the default is 0.01
points:<nderiv> REFINE estimates the derivative by calculating the cost function at either: the 3 points p-h; p; p+h or at the 5 points p-2h; p-h; p; h+h; p+2h. The five point estimate is more accurate and is recommended for parameters like nanoparticle diameters. Default: points:3 For FWHM parameters "P_u", "P_v", "P_w" the default is 5

## A.9 reset

```
reset
```

Puts the REFINE section back into the state at system start.

## A.10 run

```
run <work.mac> [, "plot:"<kuplot_macro.mac>]
```

Starts the actual refinement using the macro <work.mac> to simulate the structure and to evaluate the cost function.
The optional parameter "plot:" instructs REFINE to display the observed and calculated data at each cycle. The macro <kuplot_macro> must be a pure KUPLOT macro. The last line in this macro must be an 'exit' command that returns to REFINE.

## A.11 show

```
show
```

Shows the current refinement status.

## A.12 set

```
set cycle,<maxc>
set conver, ["status:on" | "status:off"]
set conver ["dchi:"<delta>] [,"pshift:"<max>] [,"conf:"<level>]
           ["chisq:<value>] [,"lambda:"<max>]
set relax  ["start:"<lamda_s>] [,"fail:"<lamda_f>]
           [,"success:"<lamda_g>]
set log "on"| "off"
```

Several basic settings to fine tune the refinement. These can be typed separately or combined on a single command line.

### cycle

```
set cycle,<maxc>
```

Sets the maximum number of refinement cycles

### convergence

```
set conver [,"dchi:"<delta>] [,"pshift:"<max>] [,"conf:"<level>]
           [,"chi2:"<level>] [,"lambda:"<max>]
           [,"status:on" | "status:off"]
```

REFINE will test four convergence criteria after each cycle and will stop if any of these criterial is met. Allows the user to define convergence criteria.
If the status is set to "on", the convergence criteria are used. Otherwise the refinement will run for the cycles defined by ==> 'set cycle'.
dchi:<delta> If the value of Chi**2/(Ndata-Npara) decreases by less than <delta> convergence is reached. Defaults to 0.5 pshift:<max> If all refinement parameters change by less then |DeltaP/Sigma| convergence is reached. Defaults to 0.005 conf:<level> If the confidence level is greater than <level> convergence is reached. Defaults to 0.010 chi2:<level> If the value of Chi**2/(Ndata-Npara) falls below <level> convergence is reached. Defaults to 0.500 lambda:<max> If the internal control variable lambda is larger than <max> convergence is reached. Defaults to 1.0*10**10
Convergence is reached, if: Either one of the three conditions are met. Criterion 1: (dchi AND pshift AND conf) Largest parameter shift divided by the corresponding sigma is less than pshift:<max> AND
The confidence level is larger than conf:<level> AND
The change in Chi**2 is less than dchi:<delta>
Criterion 2: (dchi AND pshift>0) The change in Chi**2 is less than dchi:<delta> AND
Any parameter changed significantly
Criterion 3: chi2 Chi**2 is less than chisq:<value>
Criterion 4: lambda Lambda is larger than lambda:<max>

### relax

```
set relax  ["start:"<lamda_s>] [,"fail:"<lamda_f>]
           [,"success:"<lamda_g>]
```

At each refinement cycle REFINE will change the parameter values in order to move towards the minimum of Chi**2. As long as the refinement is far from the optimum values, large parameter steps ensure a faster convergence. Close to the optimum values, smaller steps will ensure that the optimum values are reached without accidentally stepping beyond. An internal value "lambda" fine tunes this behavior, a larger value of "lambda" corresponds to smaller parameter changes and vice versa. REFINE will adjust these values automatically and usually you can rely on the default values. In rare cases you might have to fine tune.
"start:"<lamda_s> Defines the initial "lambda" value, Default is 0.001
"success:"<lamda_g> If the last refinement cycle improved Chi**2, "lambda" is multiplied by lamda_g, which should be less than 1 to take larger parameter steps in the next cycle. Default is 0.5
"fail:"<lamda_f> If the last cycle did not improve Chi**2, the parameter step were probably too large, and REFINE steped beyond the Chi**2 minimum. With lamda_f > 1 the lambda value is increased to take smaller steps. Default is 4.0

### log

```
set log, on
set log, off
```

If refine is run in parallel vie mpiexec (MAC, LINUX) or the suite/parallel caommand(WINDOWS) you can create log files that document the run of the work macro. These logs will be stored in an otherwise temporary directory called "DISCUS_SUITE_DERIVATIVES" located within the current directory. Logfils are called "LOGFILE.nnnn", where nnnn is a number that enumerates the calculations needed for all derivatives.

## A.13 sigma

```
sigma "kuplot", <number>
sigma <type>,<infile>
sigma "kuplot", <number>
```

The calculation will refine parameters versus the observed data and use sigmas that are stored in the KUPLOT data set number <number>

```
sigma <type>,<infile>
```

The calculation will refine parameters versus the observed data and use sigmas that are loaded from file <infile>. See the ==> 'kuplot/load' command for details on proper ways to load a data set.
Refer to the KUPLOT section for instructions on loading data.

# Bibliography

Levenberg. A method for the solution of certain problems in least squares. Quart. Appl. Math. **2** (1944) 164 – 168.

D. W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. J. Soc. Ind. Appl. Math. **11** (1963) 431–441.

W. H. Press; B. P. Flannery; S. A. Teukolsky; W. T. Vetterling. *Numerical Recipes; The art of Scientific Computing*. Cambridge University Press, Cambridge 1989.

K. V. Price; R. M. Storn; J. A. Lampinen. *Differential Evolution; A Practical Approach to Global Optimization*. Springer, Berlin 2005.